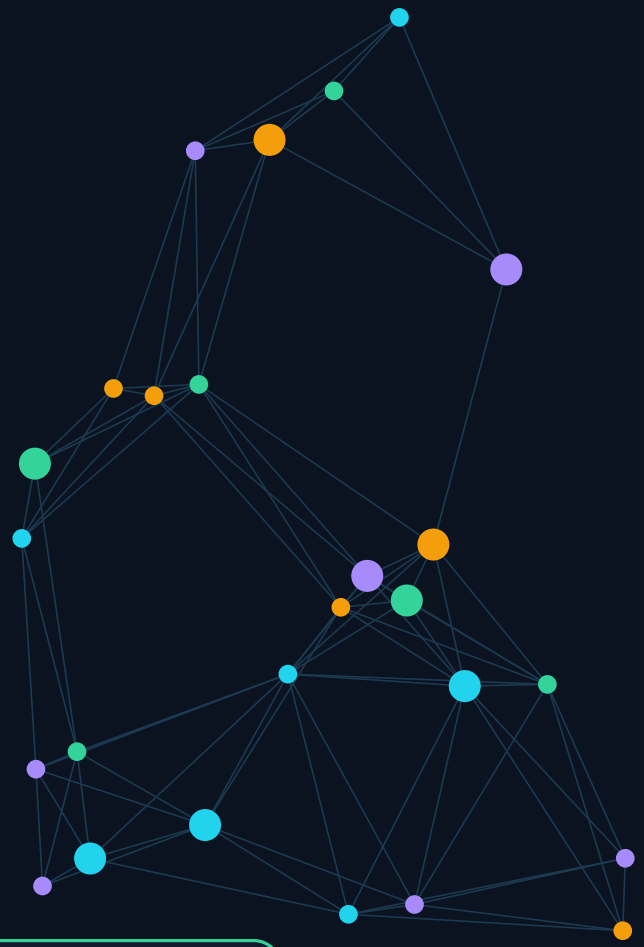
AWS | NETWORKING

Designing a High Availability VPC Architecture

A visual, step-by-step guide to building a VPC with public and private subnets, an Internet Gateway, route tables and an EC2 instance on AWS.



Documented by

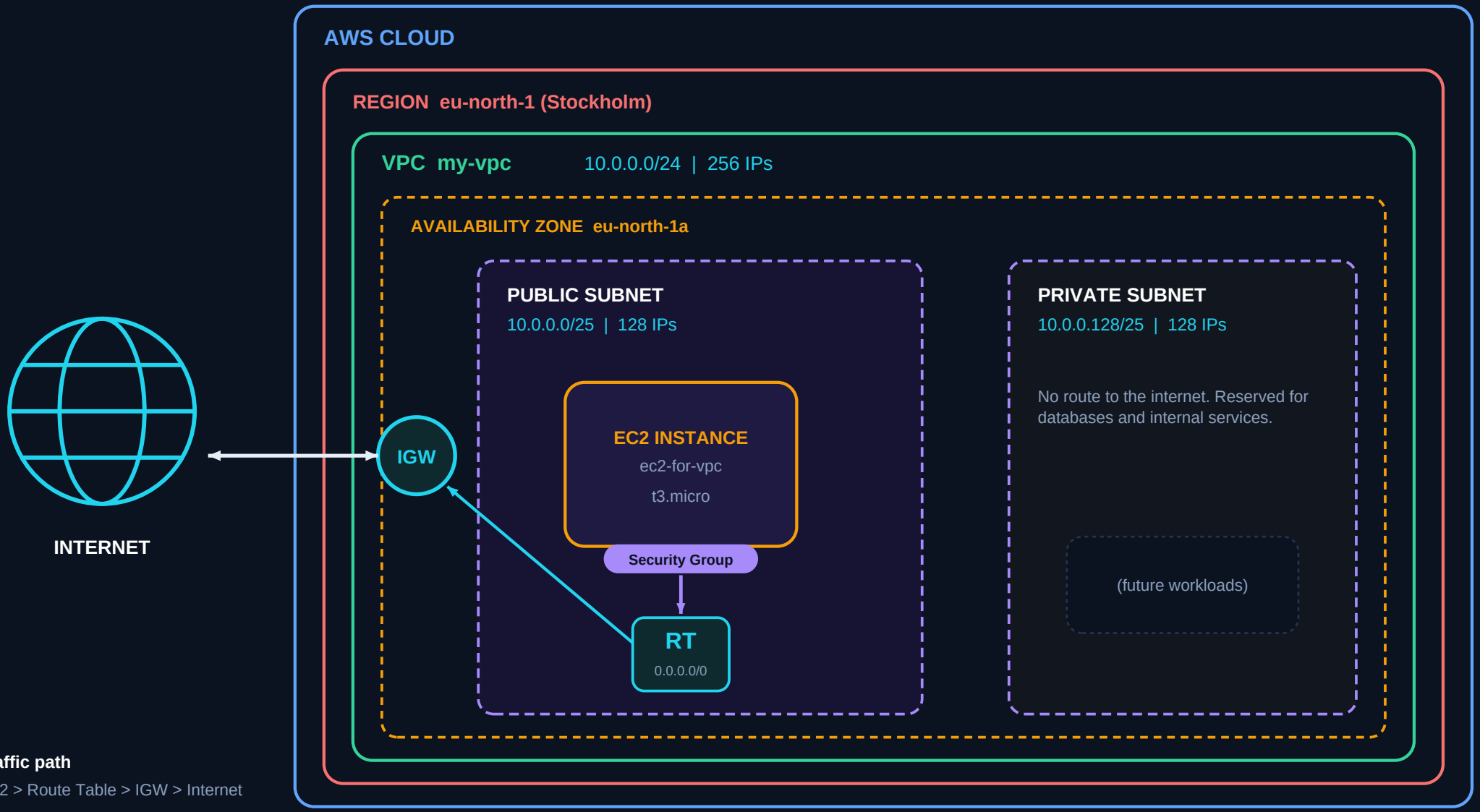
Bharathi B

LINKEDIN
https://www.linkedin.com/in/bharathib19/

GITHUB | INTERACTIVE VISUALISATION
https://bharathibalu19.github.io/vpc-visuallization/

OVERVIEW

Architecture at a Glance



Traffic path

EC2 > Route Table > IGW > Internet

GETTING STARTED

Introduction & Prerequisites

A Virtual Private Cloud (VPC) is the foundation of networking on AWS. It gives you a logically isolated section of the AWS cloud where you control IP ranges, subnets, routing and gateways.

This project builds a VPC with public and private subnets, an Internet Gateway and an EC2 instance, then proves connectivity by pinging the internet from the instance.

WHAT YOU WILL BUILD

PREREQUISITES

1. **Active AWS account**
   Free tier is enough for this project.
2. **Networking basics**
   CIDR, subnets, routing and gateways.
3. **AWS Console access**
   Comfortable navigating VPC and EC2.

PROJECT SNAPSHOT

eu-north-1	10.0.0.0/24	2 x /25	t3.micro	Amazon Linux 2023
Region	VPC CIDR	Subnets	Instance	OS

DESIGN

IP Address Plan

The VPC uses 10.0.0.0/24 (256 addresses). It is split evenly into two /25 subnets of 128 addresses each, one public and one private.

10.0.0.0/24 - 256 IPs

SUBNET	CIDR	RANGE	TOTAL IPs	USABLE
public-subnet	10.0.0.0/25	10.0.0.0 - 10.0.0.127	128	123
private-subnet	10.0.0.128/25	10.0.0.128 - 10.0.0.255	128	123

WHY 123 AND NOT 128?

AWS reserves 5 addresses in every subnet: the network address, the VPC router, the DNS server, one reserved for future use, and the broadcast address. That is why the console shows 123 available IPv4 addresses.

STEP 1 OF 6

I. Creating the VPC

1. **Log in**
   AWS Management Console
2. **Search VPC**
   Open the VPC service
3. **Dashboard**
   VPC dashboard
4. **Create VPC**
   Top-left orange button
5. **VPC only**
   Resources to create

VPC SETTINGS

Resources to create	**VPC only**
Name tag	**my-vpc**
IPv4 CIDR	**10.0.0.0/24**
IPv6 CIDR block	**No IPv6 CIDR block**
Tenancy	**Default**

Before you click Create

Use IPv4 CIDR manual input and type 10.0.0.0/24.
The Name tag is added automatically as key Name, value my-vpc.
Scroll down and click Create VPC.

SUCCESS CHECK

A green banner confirms creation. State should show Available, DNS resolution Enabled, and Default VPC No.

State: Available

STEP 2 OF 6

II. Creating Public & Private Subnets

1. **Subnets** — VPC dashboard, left menu
2. **Create subnet** — Orange button
3. **Select VPC** — Choose my-vpc
4. **Configure** — Name, AZ, CIDR
5. **Create subnet** — Repeat for second subnet

PUBLIC SUBNET

Subnet name	**public-subnet**
Availability Zone	**eu-north-1a**
VPC CIDR	**10.0.0.0/24**
Subnet CIDR	**10.0.0.0/25**
IPs	**128**

PRIVATE SUBNET

Subnet name	**private-subnet**
Availability Zone	**eu-north-1a**
VPC CIDR	**10.0.0.0/24**
Subnet CIDR	**10.0.0.128/25**
IPs	**128**

Tip: both subnets sit in the same Availability Zone here to keep the lab simple. For true high availability, mirror the pair in a second AZ such as eu-north-1b (for example 10.0.0.0/26 style splits or a larger VPC CIDR).

STEP 3 OF 6

III. Setting up the Internet Gateway

1. **Open IGWs**
   Left panel, Internet gateways
2. **Create IGW**
   Click Create internet gateway
3. **Name it**
   Name tag: IGW
4. **Attach to VPC**
   Select my-vpc
5. **Verify**
   State = Attached

BEFORE AND AFTER ATTACHING

Detached: no internet

Attached: ready to route

STEP 4 OF 6

IV. Configuring Route Tables

1. **Route tables** — Left panel, new tab
2. **Create** — Name: rt, VPC: my-vpc
3. **Associate** — Subnet associations tab
4. **Edit routes** — Add route
5. **Save** — 0.0.0.0/0 to IGW

ROUTE TABLE rt

DESTINATION	TARGET	STATUS
10.0.0.0/24	local	Active
0.0.0.0/0	igw-xxxxxxxx (IGW)	Active

SUBNET ASSOCIATIONS

BEST PRACTICE: KEEP PRIVATE SUBNETS PRIVATE

The original walkthrough associated both subnets with the route table that has the 0.0.0.0/0 route to the Internet Gateway. Any subnet using that table becomes a public subnet. To keep the private subnet truly private, associate only public-subnet with rt and leave private-subnet on the Main route table, which has only the local route.

Result: public-subnet can reach the internet through the IGW, while private-subnet stays reachable only from inside the VPC.

STEP 5 OF 6

V. Launching an EC2 Instance

1. **Open EC2** — Search EC2, new tab
2. **Launch instance** — Orange button
3. **Name + AMI** — ec2-for-vpc, Amazon Linux
4. **Key pair** — Create or reuse
5. **Network** — my-vpc, public subnet

INSTANCE

Name	**ec2-for-vpc**
AMI	**Amazon Linux 2023**
Instance type	**t3.micro (free tier)**
Key pair	**new-key-for-vpc**
Storage	**1 volume, 8 GiB**

NETWORK

VPC	**my-vpc (10.0.0.0/24)**
Subnet	**public-subnet**
Auto-assign public IP	**Enable**
Security group	**Create new**
Inbound rule	**SSH (22)**

Security tip: the wizard defaults SSH to Anywhere (0.0.0.0/0). For anything beyond a short lab, change the source to My IP. The instance must be in the public subnet with a public IP for the ping test to work.

STEP 5 CONTINUED

Connect & Test Connectivity

1. **Wait**
   3/3 status checks passed
2. **Select + Connect**
   Top of Instances page
3. **Instance Connect**
   User: ec2-user
4. **Run ping**
   ping 8.8.8.8

EC2 Instance Connect - ec2-for-vpc

```
[ec2-user@ip-10-0-0-242 ~]$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=55 time=2.5 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=55 time=2.5 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=55 time=2.5 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=55 time=2.5 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=55 time=2.5 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=55 time=2.5 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=55 time=2.5 ms
^C
--- 8.8.8.8 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss
rtt min/avg/max = 2.50/2.51/2.52 ms
[ec2-user@ip-10-0-0-242 ~]$
```

What success looks like

Replies from 8.8.8.8 and 0% packet loss.

If ping fails

Check the 0.0.0.0/0 route, IGW state, public IP, and subnet association.

WHY IT WORKS

EC2 > subnet route table > IGW > internet, and replies return the same way.

STEP 6 OF 6

VI. Deletion & Clean-up

Remove resources when finished so nothing keeps running. Delete the instance first, then the VPC.

1 Terminate the EC2 instance

Select ec2-for-vpc, then Instance state, then Terminate (delete). The root EBS volume is deleted with it. Termination cannot be undone.

Shutting-down

2 Delete the VPC

Actions, then Delete VPC. Type delete to confirm. AWS also removes the attached resources listed below.

RESOURCES DELETED WITH THE VPC

- Internet Gateway
- Route table (rt)
- Security group
- private-subnet
- public-subnet

Check the EC2 Instances list shows Terminated and the VPC list no longer shows my-vpc. An empty console means no further charges from this project.

WRAP-UP

Conclusion & Next Steps

This project walked through building a VPC from scratch on AWS: a /24 network split into public and private /25 subnets, an Internet Gateway, a custom route table and a working EC2 instance verified with ping. It is the foundation for scalable and resilient cloud applications.

TAKE IT FURTHER

Add a second AZ

Duplicate both subnets in eu-north-1b for real high availability.

NAT Gateway

Give private subnets outbound-only internet access.

Load balancer + Auto Scaling

Spread traffic across instances in both AZs.

VPC Flow Logs

Monitor and troubleshoot network traffic.

CONNECT & EXPLORE

Bharathi B

LINKEDIN

https://www.linkedin.com/in/bharathib19/

GITHUB | INTERACTIVE VISUALISATION

https://bharathibalu19.github.io/vpc-visuallization/

Make your infrastructure as resilient as your ambitions.